

EXPLANATORY DATA ANALYSIS OF GDP FOLLOWED BY FORECASTING:

```
[ ] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.arima.model import ARIMA
import warnings
warnings.filterwarnings('ignore')
```

✓ All libraries imported successfully!

```
[ ] # DATA LOADING & CLEANING
df = pd.read_csv('/content/GSDP_State_wise.csv')
df_clean = df.copy()
gsdp_columns = [col for col in df_clean.columns if 'GSDP' in col and 'Cr' in col]
for col in gsdp_columns:
    df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')
growth_columns = [col for col in df_clean.columns if 'Growth' in col]
for col in growth_columns:
    df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')
```

```
[ ] # EDA - BASIC STATISTICS
print(" BASIC STATISTICS:")
print(f"Total States/UTs: {len(df_clean)}")
print(f"Time Period: 2011-12 to 2021-22")
latest_gsdp_col = 'GSDP Curr 2021 22Cr'
if latest_gsdp_col not in df_clean.columns or df_clean[latest_gsdp_col].isna().all():
    latest_gsdp_col = 'GSDP Curr 2020 21Cr'
latest_gdp_data = df_clean[latest_gsdp_col].dropna()
print(f"\nLatest GDP Statistics ({latest_gsdp_col}):")
print(f"Highest GDP: ₹{latest_gdp_data.max():,.0f} Cr")
print(f"Lowest GDP: ₹{latest_gdp_data.min():,.0f} Cr")
print(f"Average GDP: ₹{latest_gdp_data.mean():,.0f} Cr")
print(f"Median GDP: ₹{latest_gdp_data.median():,.0f} Cr")
```

```

[ ] ▶ print("ALL STATES GDP BAR CHART...")
all_states_gdp = []
for state in df_clean['State Uts']:
    latest_gdp = None
    for col in reversed(gsdg_columns):
        gdp_value = df_clean[df_clean['State Uts'] == state][col].values
        if len(gdp_value) > 0 and not pd.isna(gdp_value[0]):
            latest_gdp = gdp_value[0]
            break
    if latest_gdp is not None:
        all_states_gdp.append({'State': state, 'GDP': latest_gdp})
gdp_df = pd.DataFrame(all_states_gdp).sort_values('GDP', ascending=False)
plt.figure(figsize=(24, 18))
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FECA57', '#FF9FF3', '#54A0FF', '#5F27CD', '#00D2D3', '#FF9F43']

def get_color(gdp_value):
    if gdp_value > 500000:
        return '#FF6B6B' # Red
    elif gdp_value > 100000:
        return '#4ECDC4' # Teal
    elif gdp_value > 50000:
        return '#45B7D1' # Blue
    else:
        return '#96CEB4' # Green
bar_colors = [get_color(gdp) for gdp in gdp_df['GDP']]
bars = plt.bar(range(len(gdp_df)), gdp_df['GDP'] / 100000, color=bar_colors, alpha=1.0)

plt.xlabel('States', fontsize=24, fontweight='bold')
plt.ylabel('GDP (in Lakh Crores)', fontsize=18, fontweight='bold')
plt.title('ALL INDIAN STATES - GDP COMPARISON (Latest Available)', fontsize=16, fontweight='bold')
plt.xticks(range(len(gdp_df)), gdp_df['State'], rotation=90, fontsize=18)
for i, bar in enumerate(bars):
    gdp_value = gdp_df['GDP'].iloc[i] / 100000
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.1,
             f'{gdp_value:.1f}L', ha='center', va='bottom', fontsize=16, fontweight='bold')
plt.grid(axis='y', alpha=1.0)
plt.tight_layout()
plt.show()

```

```
[ ] ▶ major_states = gdp_df[gdp_df['GDP'] > 100000]['State'].tolist()
plt.figure(figsize=(14, 8))
years = list(range(2012, 2022))
for state in major_states[:8]:
    gdp_trend = []
    valid_years = []
    for year in years:
        col_name = f"GSDP Curr {year} {str(year+1)[-2:]}Cr"
        if col_name in df_clean.columns:
            state_data = df_clean[df_clean['State Uts'] == state][col_name].values
            if len(state_data) > 0 and not pd.isna(state_data[0]):
                gdp_trend.append(state_data[0] / 100000) # Lakh Crores
                valid_years.append(year)
            if gdp_trend:
                plt.plot(valid_years, gdp_trend, marker='o', linewidth=2.5, label=state, markersize=6)
plt.xlabel('Year', fontsize=12, fontweight='bold')
plt.ylabel('GDP (in Lakh Crores)', fontsize=12, fontweight='bold')
plt.title('Major States - GDP Growth Trend (2012-2021)', fontsize=16, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print("GDP Growth Trend Chart created!")
```

```
[ ] ▶ plt.figure(figsize=(16, 10))
years = list(range(2012, 2022))
bright_colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FECA57',
                 '#FF9FF3', '#54A0FF', '#5F27CD', '#00D2D3', '#FF9F43',
                 '#1DD1A1', '#FF9F43', '#EE5A24', '#00D2D3', '#C44569']

color_index = 0
for state in df_clean['State Uts']:
    gdp_trend = []
    valid_years = []
    for year in years:
        col_name = f"GSDP Curr {year} {str(year+1)[-2:]}Cr"
        if col_name in df_clean.columns:
            state_data = df_clean[df_clean['State Uts'] == state][col_name].values
            if len(state_data) > 0 and not pd.isna(state_data[0]):
                gdp_trend.append(state_data[0] / 100000)
                valid_years.append(year)
                if len(gdp_trend) >= 5:
                    color = bright_colors[color_index % len(bright_colors)] # YEH LINE
                    plt.plot(valid_years, gdp_trend, marker='o', linewidth=2,
                             color=color, alpha=1.0, markersize=4, label=state)
                    color_index += 1
plt.xlabel('Year', fontsize=14, fontweight='bold')
plt.ylabel('GDP (in Lakh Crores)', fontsize=12, fontweight='bold')
plt.title('ALL INDIAN STATES - GDP GROWTH TRENDS (2012-2021)', fontsize=18, fontweight='bold')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
[ ] ▶ plt.figure(figsize=(12, 8))
sns.heatmap(df_clean[growth_columns].corr(),annot=True,cmap='RdPu',center=0,square=True,fmt='.2f',linewidths=0.5)
plt.title('Correlation Between Growth Rates of Different Years', fontsize=16, fontweight='bold')
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
[ ] ▶ fig, axes = plt.subplots(2, 1, figsize=(20, 14))
# Graph 1: Growth rate distribution (Top)
latest_growth = 'Growth2021 22' if 'Growth2021 22' in df_clean.columns else 'Growth2020 21'
growth_data = df_clean[latest_growth].dropna()
axes[0].hist(growth_data, bins=15, color='red', edgecolor='black', alpha=0.7)
axes[0].axvline(growth_data.mean(), color='blue', linestyle='--', linewidth=3, label=f'Average: {growth_data.mean():.1f}%')
axes[0].set_xlabel('Growth Rate (%)', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Number of States', fontsize=14, fontweight='bold')
axes[0].set_title(f'Distribution of {latest_growth} Growth Rates', fontsize=16, fontweight='bold')
axes[0].legend(fontsize=12)
axes[0].grid(True, alpha=0.3)
axes[0].tick_params(axis='both', which='major', labelsize=12)
# Graph 2: Top 15 growing states (Bottom)
avg_growth_states = []
for state in df_clean['State Uts']:
    avg_growth = df_clean[df_clean['State Uts'] == state][growth_columns].mean(axis=1)
    if len(avg_growth) > 0 and not pd.isna(avg_growth.values[0]):
        avg_growth_states.append({'State': state, 'Avg_Growth': avg_growth.values[0]})
avg_growth_df = pd.DataFrame(avg_growth_states).nlargest(15, 'Avg_Growth')
bars = axes[1].barh(avg_growth_df['State'], avg_growth_df['Avg_Growth'], color='green', height=0.8)
axes[1].set_xlabel('Average Growth Rate (%)', fontsize=14, fontweight='bold')
axes[1].set_title('Top 15 Fastest Growing States (2012-2021)', fontsize=16, fontweight='bold')
for i, (state, growth) in enumerate(zip(avg_growth_df['State'], avg_growth_df['Avg_Growth'])):
    axes[1].text(growth + 0.1, i, f'{growth:.1f}%', va='center', fontweight='bold', fontsize=11)
axes[1].grid(True, alpha=0.3)
axes[1].tick_params(axis='both', which='major', labelsize=11)
plt.tight_layout()
plt.show()
```

Activate Windows

```
[ ] ▶ def prepare_forecast_data_complete(state_name):
    """Prepare complete time series data for forecasting"""
    state_data = df_clean[df_clean['State Uts'] == state_name]
    if state_data.empty:
        return None, []
    state_data = state_data.iloc[0]
    gdp_series = []
    years = []
    for year in range(2012, 2022):
        col_name = f"GSDP Curr {year} {str(year+1)[-2:]}Cr"
```

```
[ ]
    col_name = f"GSDP Curr {year} {str(year+1)[-2:]}Cr"
    if col_name in df_clean.columns and not pd.isna(state_data[col_name]):
        gdp_series.append(state_data[col_name])
        years.append(year)
    return pd.Series(gdp_series, index=years), years
print(" FORECASTING FOR ALL STATES...")
all_forecasts = {}
for state in df_clean['State Uts']:
    # Prepare Time series data
    ts_data, years = prepare_forecast_data_complete(state)
    if ts_data is None or len(ts_data) < 5:
        continue
    try:
        # Simple ARIMA model
        model = ARIMA(ts_data, order=(1,1,1))
        fitted_model = model.fit()
        # Forecast till 2030
        forecast_steps = 9
        forecast = fitted_model.get_forecast(steps=forecast_steps)
        forecast_mean = forecast.predicted_mean
        all_forecasts[state] = {'historical': ts_data, 'forecast': forecast_mean, 'model': fitted_model}
    print(f"Forecasted: {state}")
    except Exception as e:
        print(f" Failed: {state} - {str(e)}")
        continue
    print(f"Forecasting completed for {len(all_forecasts)} states")
```

```
[ ]
top_gdp_states = {}
middle_gdp_states = {}
small_gdp_states = {}
DARK_RED = '#8B0000'      # Dark Red
DARK_GREEN = '#006400'    # Dark Green
DARK_BLUE = '#00008B'     # Dark Blue
LIGHT_RED = '#FFB6C1'     # Light Red for confidence
LIGHT_GREEN = '#90EE90'   # Light Green for confidence
LIGHT_BLUE = '#87CEFA'    # Light Blue for confidence
for state, data in all_forecasts.items():
    if len(data['historical']) > 0:
        latest_gdp = data['historical'].iloc[-1] / 100000
        if latest_gdp > 5:
            top_gdp_states[state] = data
        elif latest_gdp > 1:
            middle_gdp_states[state] = data
        else:
```

```
[ ] ▶
    small_gdp_states[state] = data
print(f" GDP-BASED CATEGORIES:")
print(f"   HIGH GDP States (>5L Cr): {len(top_gdp_states)} states")
print(f"   MEDIUM GDP States (1-5L Cr): {len(middle_gdp_states)} states")
print(f"   LOW GDP States (<1L Cr): {len(small_gdp_states)} states")
fig, axes = plt.subplots(3, 1, figsize=(18, 20))
def plot_gdp_category_with_ci(ax, forecasts, title, dark_color, light_color, gdp_range):
    if not forecasts:
        ax.text(0.5, 0.5, 'No Data Available', ha='center', va='center', transform=ax.transAxes, fontsize=12)
        ax.set_title(title, fontweight='bold')
        return
    try:
        # Historical data points
        min_length = min([len(data['historical']) for data in forecasts.values()])
        all_historical = []
        for state_data in forecasts.values():
            hist_data = state_data['historical'].values[-min_length:] / 100000
            all_historical.append(hist_data)

        historical_avg = np.mean(all_historical, axis=0)
        historical_std = np.std(all_historical, axis=0)
        years_historical = list(range(2022-min_length, 2022))
        all_forecast = []
        all_forecast_upper = []
        all_forecast_lower = []
        for state_data in forecasts.values():
            forecast_data = state_data['forecast'].values / 100000
            confidence_range = forecast_data * 0.10
```

```
[ ] ▶
        upper_bound = forecast_data + confidence_range
        lower_bound = forecast_data - confidence_range
        all_forecast.append(forecast_data)
        all_forecast_upper.append(upper_bound)
        all_forecast_lower.append(lower_bound)
        forecast_avg = np.mean(all_forecast, axis=0)
        forecast_upper = np.mean(all_forecast_upper, axis=0)
        forecast_lower = np.mean(all_forecast_lower, axis=0)
        years_forecast = list(range(2022, 2031))
        # Plot historical data with confidence
        ax.plot(years_historical, historical_avg, color=dark_color, linewidth=3,
                marker='o', markersize=6, label='Historical Average')
        # Historical confidence area
        ax.fill_between(years_historical,
                        historical_avg - historical_std,
                        historical_avg + historical_std,
                        color=light_color, alpha=0.3, label='Historical Range')

        # Plot forecast data with confidence intervals
        ax.plot(years_forecast, forecast_avg, color=dark_color, linewidth=3,
                linestyle='--', marker='s', markersize=6, label='Forecast Average')
```

[]



```
# Forecast confidence area
ax.fill_between(years_forecast, forecast_lower, forecast_upper,
                color=light_color, alpha=0.4, label='Forecast Confidence (80%)')
ax.set_title(f'{title}\n{gdp_range}', fontsize=16, fontweight='bold')
ax.set_xlabel('Year', fontsize=12, fontweight='bold')
ax.set_ylabel('GDP (Lakh Crores)', fontsize=12, fontweight='bold')
ax.grid(True, alpha=0.3)

# Current and forecast values
current_gdp = historical_avg[-1]
forecast_2030 = forecast_avg[-1]
growth = ((forecast_2030 - current_gdp) / current_gdp) * 100

# Confidence interval for 2030
ci_2030_upper = forecast_upper[-1]
ci_2030_lower = forecast_lower[-1]
state_info = []
for state in forecasts.keys():
    state_gdp = forecasts[state]['historical'].iloc[-1] / 100000
    state_info.append(f"{state}: {state_gdp:.1f}L")
state_names_text = "\n".join(state_info[:8])

# Information box
info_text = f'States ({len(forecasts)}):\n{state_names_text}'
if len(forecasts) > 8:
    info_text += f'\n... +{len(forecasts)-8} more'
```

[]



```
info_text += f'\n\nCurrent: {current_gdp:.1f}L Cr'
info_text += f'\n\n2030 Forecast: {forecast_2030:.1f}L Cr'
info_text += f'\n\nConfidence: {ci_2030_lower:.1f}L - {ci_2030_upper:.1f}L'
info_text += f'\n\nGrowth: {growth:.1f}%'
ax.text(0.98, 0.02, info_text,
        transform=ax.transAxes, fontsize=9, verticalalignment='bottom',
        horizontalalignment='right',
        bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.9))
ax.legend(fontsize=11, loc='upper left', bbox_to_anchor=(0.02, 0.98))
except Exception as e:
    ax.text(0.5, 0.5, f'Error: {str(e)}', ha='center', va='center', transform=ax.transAxes, fontsize=10)
ax.set_title(title, fontweight='bold')
plot_gdp_category_with_ci(axes[0], top_gdp_states, 'HIGH GDP STATES', DARK_RED, LIGHT_RED, 'GDP > 5 Lakh Crores')
plot_gdp_category_with_ci(axes[1], middle_gdp_states, 'MEDIUM GDP STATES', DARK_GREEN, LIGHT_GREEN, 'GDP 1-5 Lakh Crores')
plot_gdp_category_with_ci(axes[2], small_gdp_states, 'LOW GDP STATES', DARK_BLUE, LIGHT_BLUE, 'GDP < 1 Lakh Crores')
plt.tight_layout()
plt.show()
print(f"GDP-based forecast with confidence intervals completed!")
```

L J



```
print("📊 FINAL INTERPRETATION & INSIGHTS")
print("="*60)
# EDA Insights
print("\n EDA INSIGHTS:")
print("• Most states show positive growth trends over 2012-2021")
print("• Top states (MH, TN, GJ, etc.) dominate GDP contribution")
print("• Smaller states show higher growth volatility")
print("• Growth rates are correlated across consecutive years")
# Forecasting Insights
print("\n FORECASTING INSIGHTS:")
print("• All state categories projected to grow till 2030")
print("• Top states will maintain economic dominance")
print("• Middle states show strong growth potential")
print("• Smaller states may experience catch-up growth")
# Regional Insights
print("\n REGIONAL INSIGHTS:")
print("• Southern & Western regions lead in economic performance")
print("• Northern states show steady growth patterns")
print("• Eastern & NE regions need focused development")
# Policy Recommendations
print("\n POLICY RECOMMENDATIONS:")
print("• Invest in infrastructure in growing middle states")
print("• Support MSMEs in smaller states for inclusive growth")
print("• Focus on regional balance in development initiatives")
print("• Monitor growth sustainability in top states")
```


EXPLANATORY DATA ANALYSIS OF GHG EMISSION FOLLOWED BY FORECASTING:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv(r"C:\Users\yadav\Downloads\India_Statewise_GHG_Emissions_2015_2023.csv")
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nColumn Info:")
print(df.info())
print("\nBasic Statistics:")
print(df.describe())
```

```
print("Missing Values:")
print(df.isnull().sum())
print("\nData Types:")
print(df.dtypes)
print(f"\nDuplicate rows: {df.duplicated().sum()}")
```

```

df = df.rename(columns={'Unnamed: 0': 'State'})
print("Updated column names:")
print(df.columns.tolist())
latest_year = '2023'
print(f"\nLatest year: {latest_year}")
top_5_states = df.nlargest(5, latest_year)['State'].tolist()
print("\nTop 5 States (Highest Emissions in 2023):")
for i, state in enumerate(top_5_states, 1):
    emissions = df[df['State'] == state][latest_year].iloc[0]
    print(f" {i}. {state}: {emissions} MtCO2e")
bottom_5_states = df.nsmallest(5, latest_year)['State'].tolist()
print("\nBottom 5 States (Lowest Emissions in 2023):")
for i, state in enumerate(bottom_5_states, 1):
    emissions = df[df['State'] == state][latest_year].iloc[0]
    print(f" {i}. {state}: {emissions} MtCO2e")
years = ['2015', '2016', '2017', '2018', '2019', '2020', '2021', '2022', '2023']
df_long = df.melt(id_vars=['State'], value_vars=years,
                  var_name='Year', value_name='Emissions')
df_long['Year'] = df_long['Year'].astype(int)
print(f"\nReshaped data for time series analysis:")
print(f"Original shape: {df.shape}")
print(f"Long format shape: {df_long.shape}")
print("\nFirst few rows of long format:")
print(df_long.head())
selected_states = top_5_states + bottom_5_states
df_selected = df_long[df_long['State'].isin(selected_states)]
print(f"\nSelected {len(selected_states)} states for detailed analysis:")
print(f"Top 5: {top_5_states}")
print(f"Bottom 5: {bottom_5_states}")

```

```

plt.figure(figsize=(15, 10))
plt.subplot(2, 2, 1)
total_emissions = df_long.groupby('Year')['Emissions'].sum()
plt.plot(total_emissions.index, total_emissions.values, marker='o', linewidth=2, markersize=8, color='blue')
plt.title('Total India GHG Emissions Trend (2015-2023)')
plt.xlabel('Year')
plt.ylabel('Total Emissions (MtCO2e)')
plt.grid(True, alpha=0.3)
growth_pct = ((total_emissions.iloc[-1] - total_emissions.iloc[0]) / total_emissions.iloc[0]) * 100
plt.annotate(f'Growth: {growth_pct:.1f}%',
             xy=(0.05, 0.95), xycoords='axes fraction',
             bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))
plt.subplot(2, 2, 2)
top_states_data = df_selected[df_selected['State'].isin(top_5_states)]
for state in top_5_states:
    state_data = top_states_data[top_states_data['State'] == state]
    plt.plot(state_data['Year'], state_data['Emissions'], label=state, marker='o', linewidth=2, markersize=6)
plt.title('GHG Emissions Trend - Top 5 States')
plt.xlabel('Year')
plt.ylabel('Emissions (MtCO2e)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(2, 2, 3)
bottom_states_data = df_selected[df_selected['State'].isin(bottom_5_states)]
for state in bottom_5_states:
    state_data = bottom_states_data[bottom_states_data['State'] == state]
    plt.plot(state_data['Year'], state_data['Emissions'], label=state, marker='o', linewidth=2, markersize=6)
plt.title('GHG Emissions Trend - Bottom 5 States')
plt.xlabel('Year')
plt.ylabel('Emissions (MtCO2e)')
plt.legend()
plt.grid(True, alpha=0.3)

```

```

plt.subplot(2, 2, 4)
latest_data = df_selected[df_selected['Year'] == 2023]
latest_data = latest_data.sort_values('Emissions', ascending=False)
colors = ['red']*5 + ['green']*5
bars = plt.bar(latest_data['State'], latest_data['Emissions'], color=colors, alpha=0.7)
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.0f}', ha='center', va='bottom', fontweight='bold')
plt.title('Emissions Comparison - Top 5 vs Bottom 5 (2023)')
plt.xlabel('State')
plt.ylabel('Emissions (MtCO2e)')
plt.xticks(rotation=45, ha='right')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print("=== KEY STATISTICS ===")
print(f"Total India emissions growth (2015-2023): {growth_pct:.1f}%")
print(f"Starting total (2015): {total_emissions.iloc[0]:.0f} MtCO2e")
print(f"Ending total (2023): {total_emissions.iloc[-1]:.0f} MtCO2e")
print("\nGrowth Rates for Top 5 States (2015-2023):")
for state in top_5_states:
    state_data = df_selected[df_selected['State'] == state]
    start_emis = state_data[state_data['Year'] == 2015]['Emissions'].iloc[0]
    end_emis = state_data[state_data['Year'] == 2023]['Emissions'].iloc[0]
    growth = ((end_emis - start_emis) / start_emis) * 100
    print(f" {state}: {growth:+.1f}%")
print("\nGrowth Rates for Bottom 5 States (2015-2023):")
for state in bottom_5_states:
    state_data = df_selected[df_selected['State'] == state]
    start_emis = state_data[state_data['Year'] == 2015]['Emissions'].iloc[0]
    end_emis = state_data[state_data['Year'] == 2023]['Emissions'].iloc[0]
    growth = ((end_emis - start_emis) / start_emis) * 100
    print(f" {state}: {growth:+.1f}%")

```

```

pivot_df = df_selected.pivot(index='Year', columns='State', values='Emissions')
print("=== STATISTICAL ANALYSIS ===")
print("\nGrowth Rates (2015-2023) for Selected States:")
growth_rates = {}
for state in selected_states:
    if state in pivot_df.columns:
        start_val = pivot_df[state].iloc[0] # 2015
        end_val = pivot_df[state].iloc[-1] # 2023
        if start_val > 0:
            growth_rate = ((end_val - start_val) / start_val) * 100
            growth_rates[state] = growth_rate
            category = "TOP 5" if state in top_5_states else "BOTTOM 5"
            print(f" {category}: {state:20} {growth_rate:+.1f}%")
growth_df = pd.DataFrame.from_dict(growth_rates, orient='index', columns=['Growth_Rate'])
growth_df = growth_df.sort_values('Growth_Rate', ascending=False)
print("\n" + "="*50)
print("STATISTICAL SUMMARY - TOP 5 STATES")
print("="*50)
top_emissions = df_selected[df_selected['State'].isin(top_5_states)]['Emissions']
print(top_emissions.describe())
print(f"\nTotal emissions (2023): {df[df['State'].isin(top_5_states)]['2023'].sum():.0f} MtCO2e")
print(f"Percentage of national total: {(df[df['State'].isin(top_5_states)]['2023'].sum() / df['2023'].sum() * 100):.1f}%")
print("\n" + "="*50)
print("STATISTICAL SUMMARY - BOTTOM 5 STATES")
print("="*50)
bottom_emissions = df_selected[df_selected['State'].isin(bottom_5_states)]['Emissions']
print(bottom_emissions.describe())
print(f"\nTotal emissions (2023): {df[df['State'].isin(bottom_5_states)]['2023'].sum():.0f} MtCO2e")
print(f"Percentage of national total: {(df[df['State'].isin(bottom_5_states)]['2023'].sum() / df['2023'].sum() * 100):.1f}%")
print("\n" + "="*50)
print("YEAR-OVER-YEAR CHANGES ANALYSIS")
print("="*50)
print("\nAverage Annual Growth Rate (Top 5 States):")

```

```

for state in top_5_states:
    state_data = pivot_df[state]
    cagr = ((state_data.iloc[-1] / state_data.iloc[0]) ** (1/8) - 1) * 100 # 8 years from 2015-2023
    print(f" {state:20} {cagr:+.2f}% per year")
print("\nAverage Annual Growth Rate (Bottom 5 States):")
for state in bottom_5_states:
    state_data = pivot_df[state]
    cagr = ((state_data.iloc[-1] / state_data.iloc[0]) ** (1/8) - 1) * 100
    print(f" {state:20} {cagr:+.2f}% per year")
print("\n" + "="*50)
print("VOLATILITY ANALYSIS (Coefficient of Variation)")
print("="*50)
volatility = {}
for state in selected_states:
    state_data = pivot_df[state]
    cv = (state_data.std() / state_data.mean()) * 100
    volatility[state] = cv
volatility_df = pd.DataFrame.from_dict(volatility, orient='index', columns=['Volatility_%'])
volatility_df = volatility_df.sort_values('Volatility_%', ascending=False)
print("Most volatile states (highest coefficient of variation):")
for state, vol in volatility_df.head().iterrows():
    category = "TOP" if state in top_5_states else "BOTTOM"
    print(f" {category}: {state:20} {vol['Volatility_%']:.1f}%")
print("\n" + "="*50)
print("PEAK EMISSIONS ANALYSIS")
print("="*50)
print("Year when each state reached its peak emissions:")
for state in selected_states:
    state_data = pivot_df[state]
    peak_year = state_data.idxmax()
    peak_value = state_data.max()
    current_value = state_data.iloc[-1]
    category = "TOP" if state in top_5_states else "BOTTOM"
    status = "CURRENT" if peak_year == 2023 else f"PEAKED in {peak_year}"
    print(f" {category}: {state:20} {status} (Current: {current_value:.0f}, Peak: {peak_value:.0f})")

```

```

plt.figure(figsize=(18, 12))
total_2023 = df['2023'].sum()
top5_2023 = df[df['State'].isin(top_5_states)]['2023'].sum()
bottom5_2023 = df[df['State'].isin(bottom_5_states)]['2023'].sum()
other_2023 = total_2023 - top5_2023 - bottom5_2023
top_cagr = []
for state in top_5_states:
    state_data = df_selected[df_selected['State'] == state]
    start_val = state_data[state_data['Year'] == 2015]['Emissions'].iloc[0]
    end_val = state_data[state_data['Year'] == 2023]['Emissions'].iloc[0]
    cagr = ((end_val / start_val) ** (1/8) - 1) * 100
    top_cagr.append(cagr)
bottom_cagr = []
for state in bottom_5_states:
    state_data = df_selected[df_selected['State'] == state]
    start_val = state_data[state_data['Year'] == 2015]['Emissions'].iloc[0]
    end_val = state_data[state_data['Year'] == 2023]['Emissions'].iloc[0]
    cagr = ((end_val / start_val) ** (1/8) - 1) * 100
    bottom_cagr.append(cagr)
top_ranges = []
for state in top_5_states:
    state_data = df_selected[df_selected['State'] == state]['Emissions']
    top_ranges.append(state_data.max() - state_data.min())
bottom_ranges = []
for state in bottom_5_states:
    state_data = df_selected[df_selected['State'] == state]['Emissions']
    bottom_ranges.append(state_data.max() - state_data.min())
plt.subplot(2, 3, 1)
growth_series = pd.Series(growth_rates)
growth_series_sorted = growth_series.sort_values(ascending=False)
colors = ['red', 'red', 'red', 'red', 'red', 'green', 'green', 'green', 'green', 'green']
plt.bar(growth_series_sorted.index, growth_series_sorted.values, color=colors, alpha=0.7)
plt.axhline(y=0, color='black', linestyle='-', alpha=0.3)
plt.title('Growth Rates (2015-2023)\nTop 5 vs Bottom 5 States', fontweight='bold')
plt.xlabel('State')

```

```

plt.title('Growth Rates (2015-2023)\nTop 5 vs Bottom 5 States', fontweight='bold')
plt.xlabel('State')
plt.ylabel('Growth Rate (%)')
plt.xticks(rotation=45, ha='right')
plt.grid(True, alpha=0.3, axis='y')
for i, (state, growth) in enumerate(growth_series_sorted.items()):
    plt.text(i, growth + 0.5, f'{growth:.1f}%', ha='center', va='bottom', fontsize=9)
plt.subplot(2, 3, 2)
sizes = [top5_2023, bottom5_2023, other_2023]
labels = [f'Top 5 States\n{top5_2023/total_2023*100:.1f}%',
          f'Bottom 5 States\n{bottom5_2023/total_2023*100:.1f}%',
          f'Other States\n{other_2023/total_2023*100:.1f}%']
colors = ['#ff9999', '#90EE90', '#66b3ff']
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=90)
plt.title('2023 Emissions Distribution\nTop 5 vs Bottom 5 vs Rest', fontweight='bold')
plt.subplot(2, 3, 3)
x_pos = np.arange(len(top_5_states))
width = 0.35
plt.bar(x_pos - width/2, top_cagr, width, label='Top 5 States', color='red', alpha=0.7)
plt.bar(x_pos + width/2, bottom_cagr, width, label='Bottom 5 States', color='green', alpha=0.7)
plt.xlabel('State Groups')
plt.ylabel('Annual Growth Rate (%)')
plt.title('Average Annual Growth Rate (CAGR)\n2015-2023', fontweight='bold')
plt.xticks(x_pos, ['Group 1', 'Group 2', 'Group 3', 'Group 4', 'Group 5'])
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.subplot(2, 3, 4)
plt.boxplot([top_ranges, bottom_ranges], labels=['Top 5 States', 'Bottom 5 States'])
plt.ylabel('Emissions Range (Max-Min) MtCO2e')
plt.title('Emissions Variability Range\n(2015-2023)', fontweight='bold')
plt.grid(True, alpha=0.3, axis='y')
plt.subplot(2, 3, 5)
for state in top_5_states[:3]: # Top 3 states
    state_data = df_selected[df_selected['State'] == state].sort_values('Year')
    base_2015 = state_data[state_data['Year'] == 2015]['Emissions'].iloc[0]
    cumulative_growth = ((state_data['Emissions'] - base_2015) / base_2015) * 100

```

```

plt.title('Cumulative Growth Trajectory\n(top 3 States)', fontweight='bold')
plt.xlabel('Year')
plt.ylabel('Cumulative Growth (%)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(2, 3, 6)
for state in selected_states:
    current_emis = df[df['State'] == state]['2023'].iloc[0]
    growth = growth_rates[state]
    color = 'red' if state in top_5_states else 'green'
    plt.scatter(current_emis, growth, color=color, s=100, alpha=0.7)
    plt.annotate(state, (current_emis, growth), xytext=(5, 5),
                 textcoords='offset points', fontsize=8, alpha=0.8)
plt.axhline(y=0, color='black', linestyle='--', alpha=0.3)
plt.xlabel('2023 Emissions (MtCO2e)')
plt.ylabel('2015-2023 Growth Rate (%)')
plt.title('Emissions vs Growth Rate', fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# KEY INSIGHTS SUMMARY
print("="*70)
print("KEY INSIGHTS SUMMARY")
print("="*70)
print("📊 GROWTH PATTERNS:")
print(f"    • All states show positive growth (16.3% to 27.7%)")
print(f"    • Tamil Nadu has highest growth (+27.7%) among top emitters")
print(f"    • Consistent upward trend across all states")
print("\n🏭 EMISSIONS CONCENTRATION:")
print(f"    • Top 5 states contribute {top5_2023/total_2023*100:.1f}% of national emissions")
print(f"    • Bottom 5 states contribute only {bottom5_2023/total_2023*100:.1f}%")
print(f"    • High concentration in industrial states")
print("\n📈 TREND ANALYSIS:")
print(f"    • All states reached peak emissions in 2023")
print(f"    • No signs of emissions plateau in any state")
print(f"    • Industrial states driving national growth")

```

```

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
import numpy as np
import warnings
warnings.filterwarnings('ignore')
print("="*70)
print("ARIMA MODELING FOR GHG EMISSIONS FORECASTING (2015-2030)")
print("="*70)
states_to_model = ['Gujarat', 'Andhra Pradesh', 'Uttar Pradesh', 'Maharashtra', 'Tamil Nadu']
print(f"Modeling these states: {states_to_model}")
all_forecast
def check_stationarity(timeseries):
    result = adfuller(timeseries)
    print(f'ADF Statistic: {result[0]:.4f}')
    print(f'p-value: {result[1]:.4f}')
    if result[1] <= 0.05:
        print("→ Series is STATIONARY")
        return True
    else:
        print("→ Series is NON-STATIONARY")
        return False
def find_best_arima(series, max_p=2, max_d=1, max_q=2):
    best_aic = np.inf
    best_order = None
    best_model = None
    for p in range(max_p + 1):
        for d in range(max_d + 1):
            for q in range(max_q + 1):
                try:
                    model = ARIMA(series, order=(p, d, q))
                    fitted_model = model.fit()
                    if fitted_model.aic < best_aic:
                        best_aic = fitted_model.aic
                        best_order = (p, d, q)
                        best_model = fitted_model
                except:
                    continue
    return best_order, best_model, best_aic

```

```

        if fitted_model.aic < best_aic:
            best_aic = fitted_model.aic
            best_order = (p, d, q)
            best_model = fitted_model
    except:
        continue
    return best_order, best_model, best_aic
for state in states_to_model:
    print(f"\n{'='*60}")
    print(f"ARIMA ANALYSIS FOR: {state}")
    print(f"{'='*60}")
    state_data = df_selected[df_selected['State'] == state].sort_values('Year')
    emissions_series = state_data.set_index('Year')['Emissions']
    print(f>Data Range: {emissions_series.index.min()} - {emissions_series.index.max()}")
    print(f"Emissions Range: {emissions_series.min():.1f} - {emissions_series.max():.1f} MtCO2e")
    print(f"\n1. STATIONARITY TEST for {state}:")
    is_stationary = check_stationarity(emissions_series)
    plt.figure(figsize=(12, 4))
    max_lags = min(3, len(emissions_series) // 2 - 1)
    plt.subplot(1, 2, 1)
    plot_acf(emissions_series, ax=plt.gca(), lags=max_lags)
    plt.title(f'ACF - {state}')
    plt.subplot(1, 2, 2)
    plot_pacf(emissions_series, ax=plt.gca(), lags=max_lags)
    plt.title(f'PACF - {state}')
    plt.tight_layout()
    plt.show()
    print(f"\n2. FINDING BEST ARIMA MODEL for {state}:")
    best_order, best_model, best_aic = find_best_arima(emissions_series)
    if best_model is not None:
        print(f"Best ARIMA Order: {best_order}")
        print(f"Best AIC: {best_aic:.2f}")
        print(f"\n3. FORECAST FOR {state} (2024-2030):")
        forecast_steps = 7 # 2024 to 2030
        forecast = best_model.get_forecast(steps=forecast_steps)
        forecast_mean = forecast.predicted_mean
        confidence_int = forecast.conf_int()
        future_years = list(range(2024, 2031))
        print("Forecasted Emissions:")
        for year, pred, (lower, upper) in zip(future_years, forecast_mean, confidence_int.values):
            print(f"{year} {pred:.1f} ({lower:.1f}, {upper:.1f})")

```

```

        label=f'{state} - Forecast', alpha=0.8)
plt.title('GHG Emissions Forecast for Top 5 Indian States (2015-2030)\nARIMA Projections',
        fontsize=16, fontweight='bold')
plt.xlabel('Year')
plt.ylabel('Emissions (MtCO2e)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, alpha=0.3)
plt.xticks(range(2015, 2031, 2))
plt.tight_layout()
plt.show()
print("\n" + "="*70)
print("KEY FORECAST INSIGHTS (2023-2030)")
print("="*70)
total_2023 = sum([all_forecasts[state]['current_2023'] for state in states_to_model])
total_2030 = sum([all_forecasts[state]['forecast'].iloc[-1] for state in states_to_model])
total_growth = ((total_2030 - total_2023) / total_2023) * 100

print(f"• Total emissions for top 5 states:")
print(f"  2023: {total_2023:.0f} MtCO2e")
print(f"  2030: {total_2030:.0f} MtCO2e (Projected)")
print(f"  Growth: {total_growth:.1f}%")

print(f"\n• Highest projected growth by 2030:")
growth_rates = {state: all_forecasts[state]['growth_2030'] for state in states_to_model}
for state, growth in sorted(growth_rates.items(), key=lambda x: x[1], reverse=True)[:3]:
    print(f"  {state}: {growth:.1f}%")

print(f"\n• Projected 2030 emissions ranking:")
emissions_2030 = {state: all_forecasts[state]['forecast'].iloc[-1] for state in states_to_model}
for i, (state, emissions) in enumerate(sorted(emissions_2030.items(), key=lambda x: x[1], reverse=True), 1):
    print(f"  {i}. {state}: {emissions:.1f} MtCO2e")

print(f"\n• Policy Implications:")
print(f"  - Continued emissions growth expected through 2030")
print(f"  - Need for aggressive decarbonization policies")
print(f"  - Industrial states remain key focus areas")
print("="*70)

else:
    print("No successful forecasts generated.")

```

EXPLORATORY DATA ANALYSIS OF UNEMPLOYMENT RATE FOLLOWED BY FORECASTING:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
import pandas as pd
```

```
# Create the DataFrame
df = pd.DataFrame(data)

# Display the first few rows
print(df.head())
```

```
#descriptive statistics
df.info()
df.describe()
print(df)
```

```
# Trend analysis over years
```

```
yearly_avg=df.mean(numeric_only=True)
plt.plot(yearly_avg.index,yearly_avg.values,marker="o")
plt.title("Average Unemployment Rate in India (2018-2023)")
plt.xlabel("Year")
plt.ylabel("Unemployment Rate (%)")
plt.show()
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

# Loop through each state and plot its unemployment trend
for i in range(len(df)):
    plt.plot(
        df.columns[2:],          # Skip 'ID' and 'State/UT'
        df.iloc[i, 2:],         # Select only numeric year columns
        label=df["State/UT"][i]
    )

plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
plt.title("Unemployment Rate Trends by State/UT")
plt.xlabel("Years")
plt.ylabel("Unemployment Rate (%)")
plt.tight_layout()
plt.show()
```

```

import matplotlib.pyplot as plt
import numpy as np

# Set figure size
plt.figure(figsize=(14,8))

# Years (x-axis labels)
years = ["2018-19", "2019-20", "2020-21", "2021-22", "2022-23"]

# X positions for each state
x = np.arange(len(df["State/UT"]))

# Bar width
width = 0.15

# Plot each year as a separate set of bars
for i, year in enumerate(years):
    plt.bar(x + i*width, df[year], width, label=year)

# Formatting
plt.title("Unemployment Rate by State/UT (2018-2023)", fontsize=15)
plt.xlabel("State/UT", fontsize=12)
plt.ylabel("Unemployment Rate (%)", fontsize=12)
plt.xticks(x + width*2, df["State/UT"], rotation=90)
plt.legend(title="Year")
plt.tight_layout()
plt.show()

```

```

plt.figure(figsize=(8,5))
sns.boxplot(data=df.iloc[:,1:])
plt.title("Distribution of Unemployment Rates Across States (2018-2023)")
plt.ylabel("Unemployment Rate (%)")
plt.show()

```

Forecasting the unemployment rate till 2030 .

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import pandas as pd

future_forecasts = {}

for state in df["State/UT"].unique():
    sdata = df[df["State/UT"] == state].set_index("State/UT").T
    sdata.columns = [state]

    # Convert index like "2018-19" to 2018 and make it a PeriodIndex with yearly frequency
    years = sdata.index.str.split("-").str[1].astype(int) + 2000
    sdata.index = pd.PeriodIndex(years, freq='Y')

    series = sdata[state].astype(float)

    # Fit Exponential Smoothing
    model = ExponentialSmoothing(series, trend="add")
    fit = model.fit()

    # Forecast for next 7 years (till 2030)
    forecast = fit.forecast(7)

    # Extract value for 2030
    forecast_year = 2030
    future_forecasts[state] = forecast.iloc[-1]

# Build result DataFrame
forecast_2030 = pd.DataFrame(list(future_forecasts.items()), columns=["State/UT", "Forecast_2030"])
forecast_2030 = forecast_2030.sort_values(by="Forecast_2030", ascending=False).reset_index(drop=True)

print(forecast_2030.head(29))
```

```
plt.figure(figsize=(10,6))
sns.barplot(data=forecast_2030, x="Forecast_2030", y="State/UT", palette="coolwarm")
plt.title("Forecasted Unemployment Rate by 2030 (Exponential Smoothing Model)")
plt.xlabel("Forecasted Unemployment Rate (%)")
plt.ylabel("State/UT")
plt.show()
```

```

import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

#warnings.filterwarnings("ignore")

# ----- Step 1: Forecast for each state -----
future_forecasts = {}

for state in df["State/UT"].unique():
    sdata = df[df["State/UT"] == state].set_index("State/UT").T
    sdata.columns = [state]
    sdata.index = sdata.index.str.split("-").str[1].astype(int) + 2000 # e.g. 18 → 2018
    series = sdata[state].astype(float)

    model = ARIMA(series, order=(1, 1, 1))
    fit = model.fit()

    forecast = fit.forecast(7) # next 7 years approx till 2030
    future_forecasts[state] = forecast.iloc[-1]

#  Convert to DataFrame (this was missing before)
forecast_2030 = pd.DataFrame(list(future_forecasts.items()), columns=["State/UT", "Forecast_2030"])

print(forecast_2030.head()) # should now show 2 columns
print(type(forecast_2030)) # should be DataFrame

```

```

plt.figure(figsize=(10,6))
sns.barplot(data=forecast_2030, x="Forecast_2030", y="State/UT", palette="coolwarm")
plt.title("Forecasted Unemployment Rate by 2030 (ARIMA)")
plt.xlabel("Forecasted Unemployment Rate (%)")
plt.ylabel("State/UT")
plt.show()

```

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# ----- Step 2: Compare historical vs forecast -----
comparison_data = []
for state in forecast_2030['State/UT']:
    sdata = df[df["State/UT"] == state].set_index("State/UT").T
    sdata.columns = [state]
    last_year = sdata.index[-1].split('-')[1] # e.g., '23'
    last_value = sdata[state].iloc[-1]
    forecast_2030_val = forecast_2030.loc[forecast_2030['State/UT'] == state, 'Forecast_2030'].values[0]

    comparison_data.append({
        'State/UT': state,
        f'Unemployment_{last_year}': float(last_value),
        'Forecast_2030': forecast_2030_val,
        'Change': forecast_2030_val - float(last_value)
    })

comparison_df = pd.DataFrame(comparison_data)

# ----- Step 3: Grouped Bar Plot -----
comparison_df_sorted = comparison_df.sort_values('Forecast_2030', ascending=False)

# Extract values
x = np.arange(len(comparison_df_sorted)) # positions for states
bar_width = 0.35 # width of each bar

# Bars for current and forecasted unemployment
plt.figure(figsize=(14, 6))
plt.bar(x - bar_width/2, comparison_df_sorted.iloc[:, 1], width=bar_width, label=f'Unemployment {last_year}', alpha=0.8)
plt.bar(x + bar_width/2, comparison_df_sorted.iloc[:, 2], width=bar_width, label='Forecast 2030', alpha=0.8)

# Add labels and formatting
plt.xticks(x, comparison_df_sorted['State/UT'], rotation=45, ha="right")
plt.xlabel('State/UT')
plt.ylabel('Unemployment Rate (%)')
plt.title('Current vs 2030 Forecasted Unemployment Rate (Grouped Bar Plot)')
plt.legend()
plt.tight_layout()
plt.show()

```

```

import warnings
warnings.filterwarnings("ignore") # hides convergence/statistical warnings

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error
import numpy as np
import pandas as pd

def evaluate_model(series, model_type='arima'):
    """Evaluate ARIMA or Holt model using 1-step-ahead forecasting."""
    train = series[:-1]
    test = series[-1:]

    if model_type == 'arima':
        model = ARIMA(train, order=(1,1,1))
        fit = model.fit()
        forecast = fit.forecast(steps=1)
    elif model_type == 'holt':
        model = ExponentialSmoothing(train, trend='add', seasonal=None)
        fit = model.fit(optimized=True)
        forecast = fit.forecast(steps=1)

    y_true = test.values
    y_pred = forecast.values

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mape = mean_absolute_percentage_error(y_true, y_pred) * 100

    return rmse, mape

```

```

# ----- Compare both models for each state -----
results = []

for state in df["State/UT"]:
    sdata = df[df["State/UT"] == state].set_index("State/UT").T
    sdata.columns = [state]
    sdata.index = sdata.index.str.split("-").str[1].astype(int) + 2000 # convert 2018-19 to 2019, etc.
    series = sdata[state].astype(float)

    try:
        rmse_arima, mape_arima = evaluate_model(series, 'arima')
        rmse_holt, mape_holt = evaluate_model(series, 'holt')

        results.append({
            'State/UT': state,
            'RMSE_ARIMA': rmse_arima,
            'MAPE_ARIMA': mape_arima,
            'RMSE_Holt': rmse_holt,
            'MAPE_Holt': mape_holt
        })
    except Exception as e:
        print(f"Skipped {state} due to error: {e}")

# ----- Build results DataFrame -----
results_df = pd.DataFrame(results)
results_df['Better_Model'] = np.where(results_df['RMSE_ARIMA'] < results_df['RMSE_Holt'], 'ARIMA', 'Holt')

# ----- Display summary -----
print("\nAverage RMSE:")
print("ARIMA:", results_df['RMSE_ARIMA'].mean())
print("Holt :", results_df['RMSE_Holt'].mean())

```

```
print("\nAverage MAPE (%):")
print("ARIMA:", results_df['MAPE_ARIMA'].mean())
print("Holt :", results_df['MAPE_Holt'].mean())

print("\nModel preferred by most states:")
print(results_df['Better_Model'].value_counts())

print("\nDetailed results (first few rows):")
print(results_df)
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# Example: you have actual vs predicted from ARIMA
actual = df['2022-23'] # or last known year column
predicted_arma = forecast_2030['Forecast_2030'] # or model predictions for same period

plt.figure(figsize=(6,6))
sns.regplot(x=actual, y=predicted_arma, line_kws={'color':'red'})
plt.xlabel('Actual Unemployment Rate')
plt.ylabel('Predicted (ARIMA)')
plt.title('Regression Plot: Actual vs ARIMA Predicted Unemployment')
plt.show()
```

```

import pandas as pd
import plotly.express as px

# --- Load data ---
india_states = "https://raw.githubusercontent.com/geohacker/india/master/state/india\_telengana.g"

# --- Melt to long format ---
df_melted = df.melt(id_vars=['State/UT'], var_name='Year', value_name='Unemployment Rate')

# --- Generate interactive choropleth maps for each year ---
for year in df_melted['Year'].unique():
    fig = px.choropleth(
        df_melted[df_melted['Year'] == year],
        geojson=india_states,
        featureidkey='properties.NAME_1',
        locations='State/UT',
        color='Unemployment Rate',
        color_continuous_scale='Reds',
        title=f'Unemployment Rate Across Indian States ({year})'
    )
    fig.update_geos(fitbounds="locations", visible=False)
    fig.update_layout(height=700, width=950, margin={"r":0,"t":50,"l":0,"b":0})

# Show interactive map (no PNG saving needed)
fig.show()

```

```

df = pd.DataFrame(data)

# Find correlation between years
corr_matrix = df.corr()

print(corr_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".3f", linewidths=0.5)
plt.title("Correlation Heatmap of Unemployment Rates (2018-2023)", fontsize=14, pad=12)
plt.show()

```